GDEV 32 Programming Exercise 3 - Shadow Rendering

Specifications

For this programming exercise, you will build on your previous lighting exercise to render shadows using the shadow map technique discussed in class.

Tasks

1. Make a backup copy of your exercise 2 before proceeding.

2. (60 points) Study the demo8 program uploaded on Canvas. Pay specific attention to the changes made between demo5 and demo8 to implement the shadow map (enclosed in //// comment blocks). Then **adapt the shadow map code to the spotlight** on your own exercise code.
    - You are welcome to adjust the size of the shadow map by modifying the SHADOW_SIZE definition, but be careful not to set this too high, especially if you have an older GPU.
    - Also, keep in mind that you may be already using two or three texture units (for your diffuse map, normal map, and possibly specular map), but demo8 assumes that the shadow map is in the second texture unit – so change the texture bindings accordingly!

3. (15 points) Since you are using a spotlight for your exercise 2, you will want to make the following specific changes to the demo8 code:
    - Set the **field of view angle** (FOV) of the shadow map camera to match the (outer) cone angle of your spotlight. (Note that these two angles are not defined in the exact same way – what's the difference?)
    - Set the camera **eye** position to the origin of your spotlight. This way, you can change the shadowing of the scene by moving the spotlight.
    - Set the camera **center** position to the focus point of your spotlight.

    If you were not able to implement the spotlight correctly from your previous exercise, set the center position to the world origin, and just leave the field of view at 90 degrees as in demo8. (You will only get partial points if you do it this way.)

4. (10 points) Implement **keyboard controls** to turn shadows on or off.
    - To get full credit here, do not just skip the shadow check in your main fragment shader – you also need to **skip rendering the shadow map** in your main code altogether! (Think of shadows as an optional feature – your user may turn it off to speed up their game.)

5. (15 points) If you complete the above requirements perfectly, you will only get a B for this exercise. To get a B+ or A, you will need to implement **Percentage-Closer Filtering (PCF)** to render shadows with soft edges.
    - This essentially means that you should sample your shadow map multiple times per fragment, each time nudging the coordinates where you take the next sample from.
    - Your `inShadow()` function should now return a `float` (instead of a `bool`), representing the amount of shadow that is imparted on the fragment. (Also change the main fragment shader code to attenuate your diffuse and specular lighting components with this shadow amount, instead of simply turning these components off.)
    - Refer to the Kernel Effects section of our Framebuffers slide set to get an idea of how to make multiple lookups on a texture in a 3x3 grid from within your fragment shader. Note that since your shadow map is square, the "window" dimensions should also be square.
    - To make your shadows look softer (and exhibit less banding), take more samples in a larger area (instead of just a 3x3 grid) and randomize the positions where you

are taking your samples. However, try not to take more than 100 samples per (final) fragment, as any higher will significantly slow down your GPU.

- To get full credit here, implement keyboard controls to make the shadow softer or sharper -- this will involve adjusting both the size of your sampling area and the number of samples that you take within this sampling area.

Submission

1. Your submission should contain:
   - A single .cpp file.
   - Any number of support files – either shader files (.vs/.fs) or texture files (.jpg/.png).
   - A filled-in Certificate of Authorship as a .pdf.
2. Your program should expect all support files from the same folder as the executable (i.e., do not use subfolders).
3. Create a single subfolder called EXTRA and place all extraneous files there (i.e., files that you never reference in your .cpp program but were needed to create your other assets, e.g., python scripts for converting vertex data, original versions of texture files, etc.)
4. Do **not** include an .exe or .out file in your submission. I will compile the .cpp myself.
5. Do **not** include the libraries (e.g., GLAD, GLFW, GLM, stb_image.h, and gdev.h) in your submission. You may NOT use any additional external libraries for this exercise.
6. Archive all your files in a single **.zip** file (do not use other formats like .rar, .7z, or .tar.gz) and submit the .zip file via Canvas.